

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Appendix 1 Alphabetical list of FPU instructions

<u>F2XM1</u>	2 to the X power minus 1
<u>FABS</u>	Absolute value of ST(0)
<u>FADD</u>	Add two floating point values
<u>FADDP</u>	Add two floating point values and pop ST(0)
<u>FBLD</u>	Load BCD data from memory
<u>FBSTP</u>	Store BCD data to memory
<u>FCHS</u>	Change the sign of ST(0)
<u>FCLEX</u>	Clear exceptions
<u>FCMOVcc</u>	Conditional move based on CPU flags
<u>FCOM</u>	Compare ST(0) to a floating point value
<u>FCOMI</u>	Compare ST(0) to ST(i) and set CPU flags
<u>FCOMIP</u>	Compare ST(0) to ST(i) and set CPU flags and pop ST(0)
<u>FCOMP</u>	Compare ST(0) to a floating point value and pop ST(0)
<u>FCOMPP</u>	Compare ST(0) to ST(1) and pop both registers
<u>FCOS</u>	Cosine of the angle value in ST(0)
<u>FDECSTP</u>	Decrease stack pointer
<u>FDIV</u>	Divide two floating point values
<u>FDIVP</u>	Divide two floating point values and pop ST(0)
<u>FDIVR</u>	Divide in reverse two floating point values
<u>FDIVRP</u>	Divide in reverse two floating point values and pop ST(0)
<u>FFREE</u>	Free a data register
<u>FIADD</u>	Add an Integer located in memory to ST(0)
<u>FICOM</u>	Compare ST(0) to an integer value
<u>FICOMP</u>	Compare ST(0) to an integer value and pop ST(0)
<u>FIDIV</u>	Divide ST(0) by an Integer located in memory
<u>FIDIVR</u>	Divide an Integer located in memory by ST(0)
<u>FILD</u>	Load integer from memory
<u>FIMUL</u>	Multiply ST(0) by an Integer located in memory
<u>FINCSTP</u>	Increase stack pointer
<u>FINIT</u>	Initialize the FPU

<u>FIST</u>	Store integer to memory
<u>FISTP</u>	Store integer to memory and pop ST(0)
<u>FISTTP</u>	Store truncated integer to memory and pop ST(0)
<u>FISUB</u>	Subtract an Integer located in memory from ST(0)
<u>FISUBR</u>	Subtract ST(0) from an Integer located in memory
<u>FLD</u>	Load real number
<u>FLD1</u>	Load the value of 1
<u>FLDCW</u>	Load control word
<u>FLDENV</u>	Load environment
<u>FLDL2E</u>	Load the log base 2 of e (Napierian constant)
<u>FLDL2T</u>	Load the log base 2 of Ten
<u>FLDLG2</u>	Load the log base 10 of 2 (common log of 2)
<u>FLDLN2</u>	Load the log base e of 2 (natural log of 2)
<u>FLDPI</u>	Load the value of PI
<u>FLDZ</u>	Load the value of Zero
<u>FMUL</u>	Multiply two floating point values
<u>FMULP</u>	Multiply two floating point values and pop ST(0)
<u>FNCLEX</u>	Clear exceptions (no wait)
<u>FNINIT</u>	Initialize the FPU (no wait)
<u>FNOP</u>	No operation
<u>FNSAVE</u>	Save state of FPU (no wait)
<u>FNSTCW</u>	Store control word (no wait)
<u>FNSTENV</u>	Store environment (no wait)
<u>FNSTSW</u>	Store status word (no wait)
<u>FPATAN</u>	Partial arctangent of the ratio ST(1)/ST(0)
<u>FPREM</u>	Partial remainder
<u>FPREM1</u>	Partial remainder 1
<u>FPTAN</u>	Partial tangent of the angle value in ST(0)
<u>FRNDINT</u>	Round ST(0) to an integer
<u>FRSTOR</u>	Restore all registers
<u>FSAVE</u>	Save state of FPU
<u>FSCALE</u>	Scale ST(0) by ST(1)
<u>FSIN</u>	Sine of the angle value in ST(0)
<u>FSINCOS</u>	Sine and cosine of the angle value in ST(0)
<u>FSQRT</u>	Square root of ST(0)

<u>FST</u>	Store real number
<u>FSTCW</u>	Store control word
<u>FSTENV</u>	Store environment
<u>FSTP</u>	Store real number and pop ST(0)
<u>FSTSW</u>	Store status word
<u>FSUB</u>	Subtract two floating point values
<u>FSUBP</u>	Subtract two floating point values and pop ST(0)
<u>FSUBR</u>	Subtract in reverse two floating point values
<u>FSUBRP</u>	Subtract in reverse two floating point values and Pop ST(0)
<u>FTST</u>	Test ST(0) by comparing it to +0.0
<u>FUCOM</u>	Unordered Compare ST(0) to a floating point value
<u>FUCOMI</u>	Unordered Compare ST(0) to ST(i) and set CPU flags
<u>FUCOMP</u>	Unordered Compare ST(0) to ST(i) and set CPU flags and pop ST(0)
<u>FUCOMP</u>	Unordered Compare ST(0) to a floating point value and pop ST(0)
<u>FUCOMPP</u>	Unordered Compare ST(0) to ST(1) and pop both registers
<u>FWAIT</u>	Wait while FPU is busy
<u>FXAM</u>	Examine the content of ST(0)
<u>FXCH</u>	Exchange the top data register with another data register
<u>FXTRACT</u>	Extract exponent and significand
<u>FYL2X</u>	$Y * \log_2 X$
<u>FYL2XP1</u>	$Y * \log_2 (X+1)$

[RETURN TO](#)
[SIMPLY FPU](#)
[CONTENTS](#)